

Rapport de VHDL

BE gestion d'un bateau

Table des matières :

I. Introduction : **3**
II. Fonction d'entraînement **3**
A. Compteur 0-15 3
B. Compteur 0-9 avec fréquence de 1 Hz 4
C. Compteur asynchrone sans anti rebond 6
D. Compteur Asynchrone avec anti rebond 7
E. PWM 8
F. Fonction supplémentaire 10
1. Décalage à droite 10
2. Décalage contrôlé 12
III. Fonction Simple (Cap) **13**
A. Étude fonctionnelle 13
1. Interface 13
Entrées 13
Sorties 14
2. Architecture 14
3. Caractéristiques du signal de la boussole 15
4. Loi de conversion 15
5. Principe de mesure 15
6. Modes de fonctionnement 16
B. Mise en œuvre en VHDL 17
IV. Fonction complexe **18**
V. Conclusion **18**
VI. Annexe **18**

I. Introduction :

L'objectif principal du BE va être de nous introduire le langage VHDL afin de nous familiariser avec les différents aspects du logiciel quartus version 18.1 qui va nous permettre de développer les premières fonctions qui servent d'exercice jusqu'à qu'on arrive à terminer le BE. C'est-à-dire réaliser les différentes fonctions qui permettent le déplacement du bateau de type voilier automatiquement en réalisant les fonctions par exemple le cap ou encore anémomètre ainsi qu'une pwm qui permet de gérer la direction du bateau.

II. Fonction d'entraînement

Dans cette partie, nous allons avoir à faire plusieurs fonctions de plus en plus complexes afin de pouvoir se familiariser avec les différentes parties du langage VHDL. Pour ce faire, la difficulté des exercices à réaliser va augmenter de plus en plus au fur et à mesure de l'avancement du TD. Tout d'abord le premier exercice est de réaliser une fonction simple.

A. Compteur 0-15

L'objectif de la première fonction est de réaliser un compteur de 0 à 15.

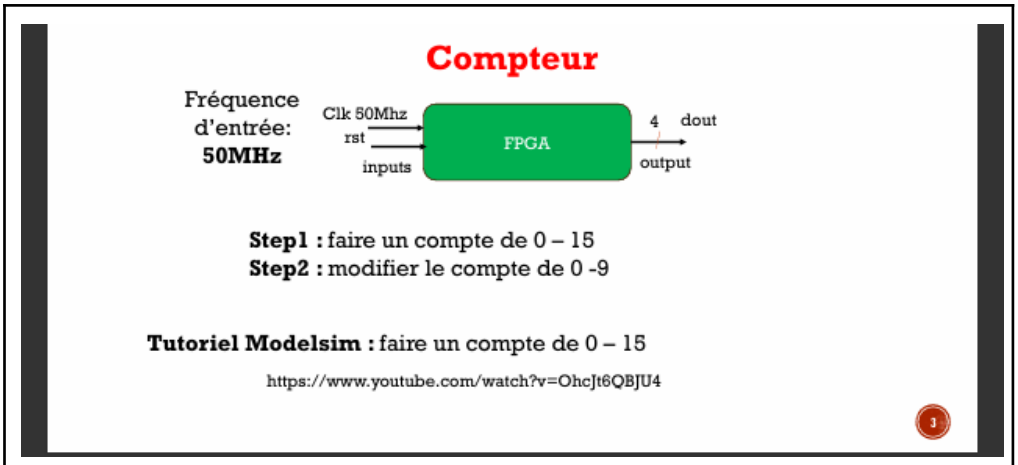


Figure 1 : Compteur 0-15

Pour réaliser un compteur de 0 à 15 qui s'incrémente toutes les 1 s avec une horloge de 50 MHz, on doit diviser l'horloge principale.

Pour obtenir une période de 1 seconde, il faut un signal d'horloge de fréquence 1 Hz. Autrement, il faut générer un signal clock de période 1 seconde.
Pour ce faire, on fait un process pour faire un signal carré de fréquence 1 Hz. Ce signal servira d'horloge pour le compteur.
Avec une horloge de 50 MHz, pour obtenir un signal de 1 Hz, on compte 25000000 fronts montants du clk 50MHz. En effet, ce temps correspond à 0,5 seconde. Ainsi, à chaque 25000000 fronts montants du clk 50MHz, on change la valeur du signal clk_1s. On obtient ainsi un signal carré de période 1 seconde, soit une fréquence de 1 Hz.
La variable clk_1s est défini comme un signal pour pouvoir l'utiliser dans un autre process pour le compteur.
Pour le compteur de 0 à 15, on utilise un vecteur de 4 bits. Ce vecteur est incrémenté à chaque front montant du signal clk_1s.
Le programme VHDL est disponible sur le repo Git sous le nom de compteur.vhd

Pour valider le bon fonctionnement du programme, nous avons effectué des simulation avec ModelSim. La période du compteur a été réduite lors de la simulation mais le principe de fonctionnement reste inchangé.
Les résultats de la simulation sont représentés sur la figure suivante.

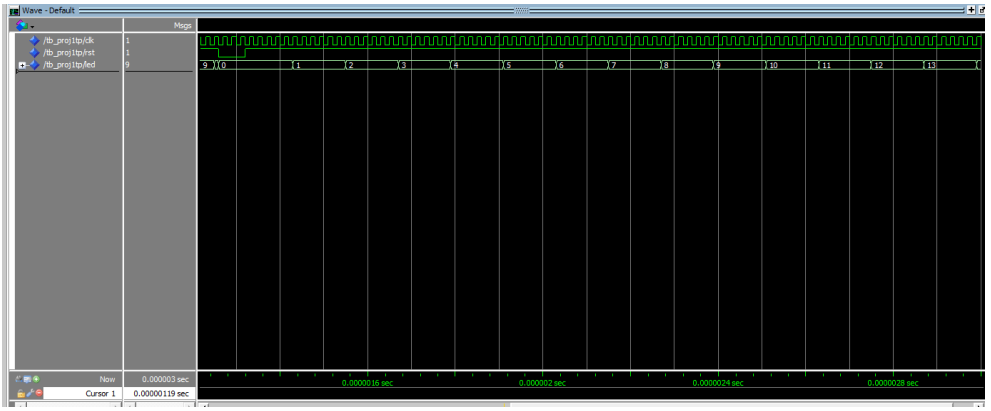


Figure 2 : Simulation du compteur 0-15

B. Compteur 0-9 avec fréquence de 1 Hz

Notre fonction de la partie, doit être un compteur qui permet d'incrémenter le compteur toutes les secondes. Il possède une sortie qui est affichée en binaire sur les 4 leds afin de pouvoir suivre et compter l'incrémentation. Nous avons en entrée rst et clk 50 MHz qui correspond à une horloge de 20ns. On va utiliser le diviseur 1 Hz afin de synchroniser les différentes fonctions. La sortie se fait sur 4 bits.

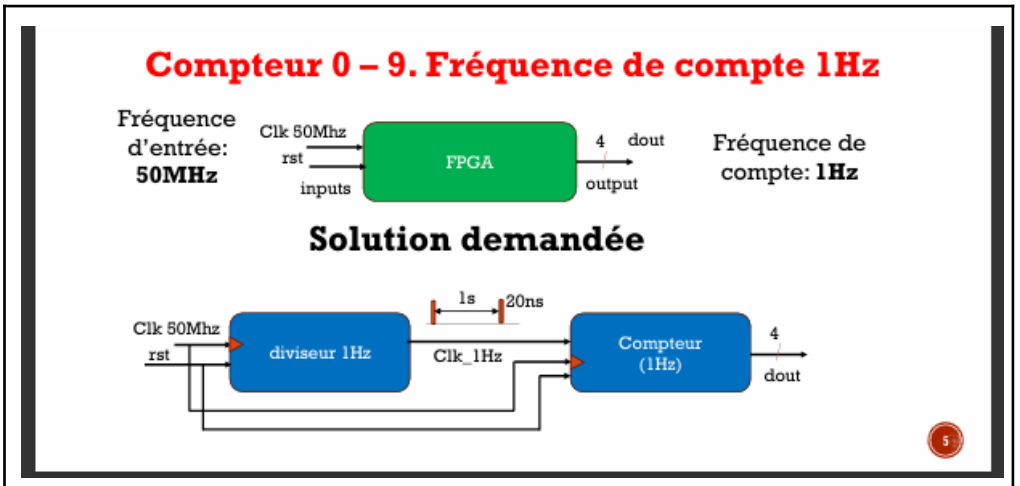


Figure 3 : Compteur 0-9, fréquence de compte 1 Hz

Afin de réussir cette partie là, nous avons réalisé dans un premier temps une simulation via un test bench qui permet de vérifier que le code effectue bien ce qui est demandé avant de le télécharger sur la carte FPGA.

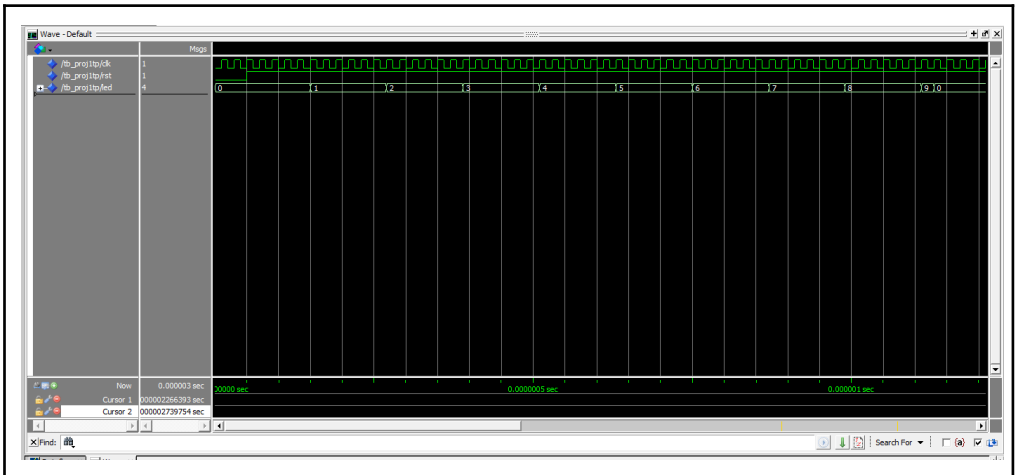


Figure 4 : Simulation de la fonction

La simulation a été accélérée cela est due au capacité de l'ordinateur portable qui était dans l'incapacité de simuler 9 secondes entières. On remarque que le compteur va s'incrémenter jusqu'à ou il revenir a 0 a l'arrivée du temps d'acquisition a la condition. La variable rst est le reset mis à 0 pour simulation afin de simuler la position du bouton quand on appuie dessus. Une fois à 1 le bouton reset est relâché.

C. Compteur asynchrone sans anti rebond

Le compteur asynchrone qui est demandé lors des travaux dirigés a été la réalisation d'un compteur asynchrone sans anti rebond.

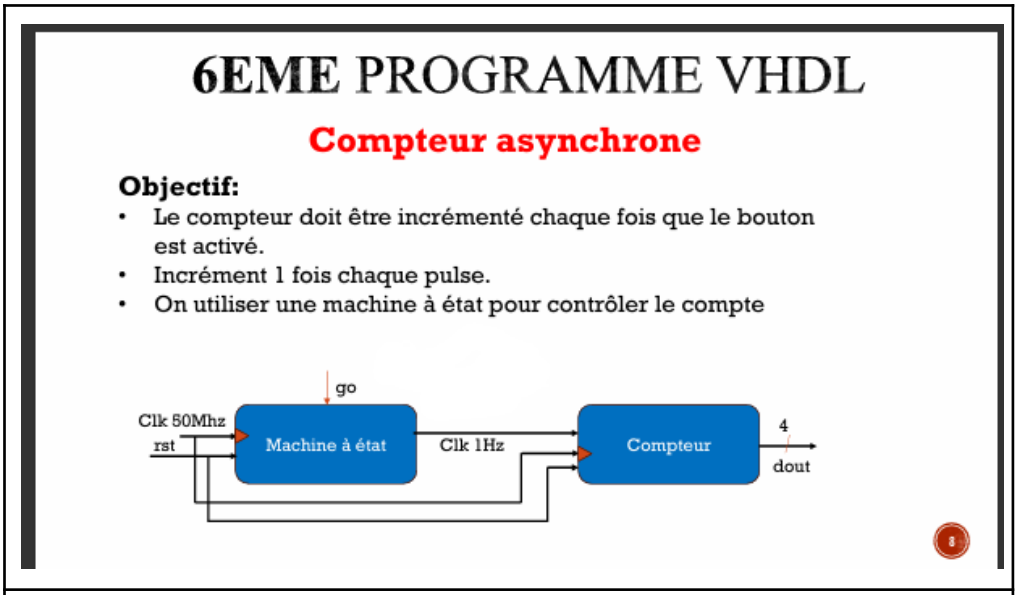


Figure 5 : Compteur asynchrone sans anti rebond

Afin de la réaliser, nous avons tout d'abord commencé par concevoir un diagramme de machine à états, observable sur la figure 6 ci-dessous. Nous avons défini deux états : Machine et Comptage. L'état initial est l'état *Machine*, qui se maintient tant qu'aucune condition d'activation n'est remplie. Dès qu'un appui sur le bouton est validé, l'état suivant bascule vers *Comptage* pour la boucle suivante, ce qui permet d'incrémenter le compteur d'une unité.

Figure 6 : Machine à état du compteur asynchrone

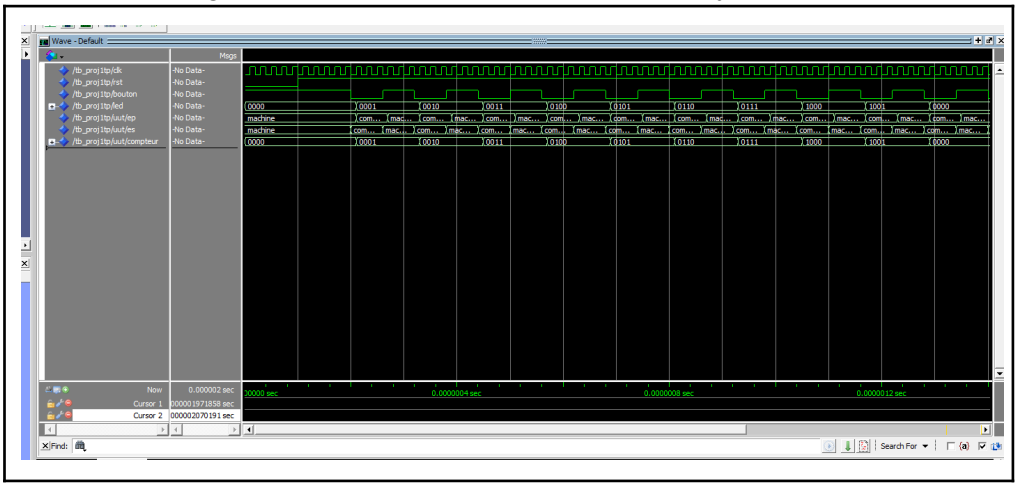


Figure 7 : Test bench

La simulation est visible sur la figure ci-dessus, montre bien le comportement de la machine à état qui a été effectué. On a pu remarquer lors des tests sur la carte que le code sans anti rebond est fonctionnel mais ce dernier saute des unités en passant par exemple 4 à 6. Cela est dû au double rebond de mécanique.

D. Compteur Asynchrone avec anti rebond

Le compteur asynchrone de cette partie va nous permettre d'éliminer le problème du rebond mécanique afin d'éviter que le compteur s'incrémente 2 fois d'affilée. Pour éviter cela, on rajoute une contrainte de 20ms à partir d'une demande d'incrémentation par bouton visible sur la figure 8.

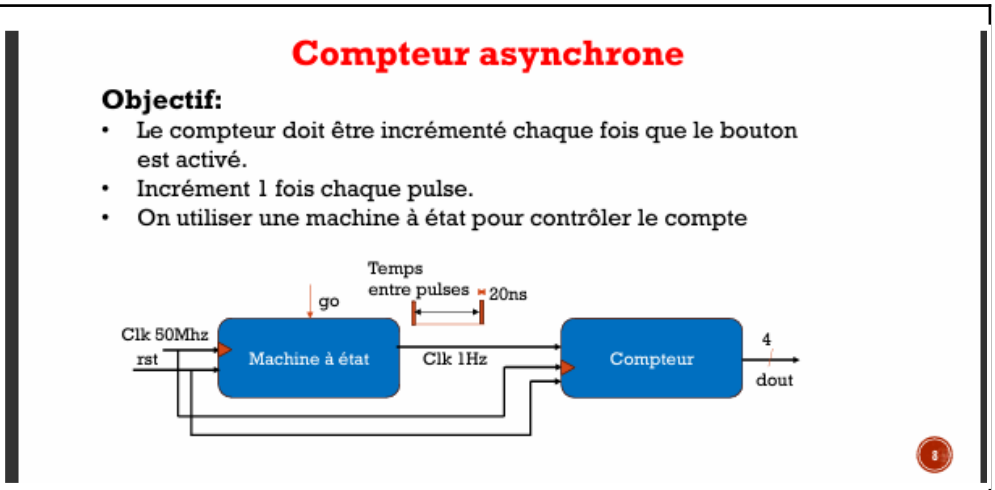


Figure 8 : Compteur asynchrone avec anti rebond

Afin de compléter la machine à états réalisée dans la partie C, nous avons ajouté un état d'attente de 20 ms. Celui-ci permet de temporiser le signal pour que chaque appui sur le bouton ne soit compté qu'une seule fois. On remarque ainsi que lors d'un appui, l'état présent devient l'état Machine ; puis, une fois le bouton relâché, l'état présent bascule vers l'état d'attente. Enfin, le cycle se termine par l'état de Comptage, qui permet d'incrémenter le compteur d'une seule unité.

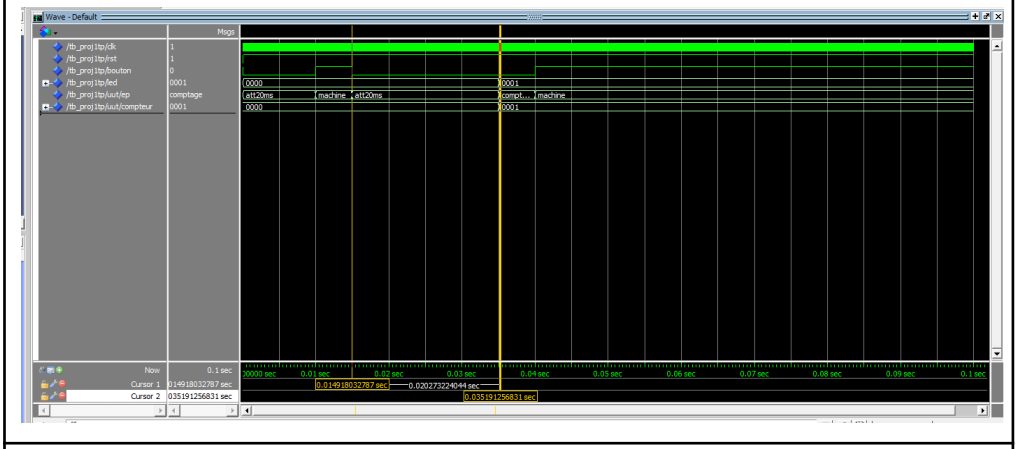


Figure 9 : Simulation

Le bouton a été mis à 1 lors de la simulation, c'est a dire a l'inverse des autres simulation. La simulation vient représenter tout ce qui a été présenté au-dessus.

E. PWM

L'objectif de cette partie est de réaliser une programme VHDL qui permet de générer un signal PWM.
Un signal PWM est un signal carré. Il est défini par une fréquence (ou période) et un rapport cyclique.
Pour faire un PWM, nous allons utiliser un compteur de 10 bits et un comparateur. Le compteur est synchronisé sur un horloge de 50 MHz soit une période d'incrémentation de 20 ns.
Ainsi, la fréquence du signal PWM est:

$$F_{PWM} = \frac{1}{2^{10} \times 20.10^{-9}} = \frac{F_{clk}}{2^{10}} = 48,8\ kHz$$

Pour obtenir une fréquence de 10 kHz, il faut diviser l'horloge de 50 MHz par 5. Ce qui donne une horloge de 10 MHz pour avoir une fréquence de 10 kHz.

$$F_{PWM} = \frac{F_{clk10MHz}}{2^{10}} = \frac{10.10^{6}}{2^{10}} = 9,765\ kHz \simeq 10\ kHz$$

La sortie du compteur est comparée au rapport cyclique défini sur 10 bits.
Si la valeur du compteur est inférieure à celle du rapport cyclique, le signal PWM est à l'état haut.
Si la valeur du compteur est supérieure à celle du rapport cyclique, le signal PWM est à l'état bas.
La figure suivante illustre la description précédente.

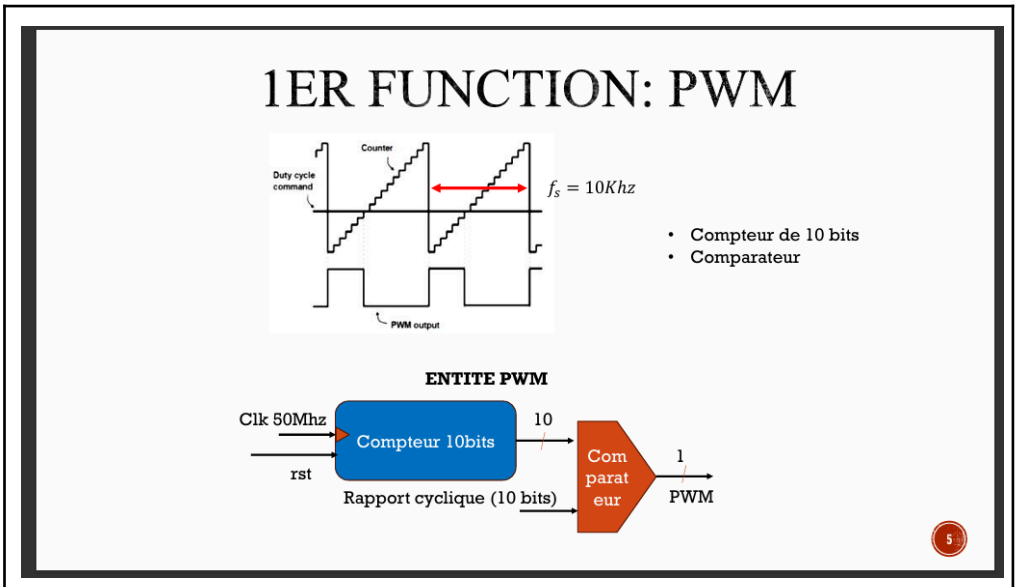


Figure 10 : Fonction PWM

Pour implémenter l'algorithme permettant de générer un signal PWM, nous avons utilisé trois "process":

- "process" qui permet de varier le rapport cyclique. Ce process permet de changer le rapport cycle à chaque appui sur un bouton.
- "process" pour faire le compteur.
- un dernier "process" pour comparer la valeur du compteur et le rapport cyclique. La sortie du comparateur est à "1" si la valeur du compteur est inférieur au rapport cyclique, à "0" sinon.

Le programme VHDL est disponible sur le repo Git.
Le résultat de la simulation est représenté à la figure suivante.

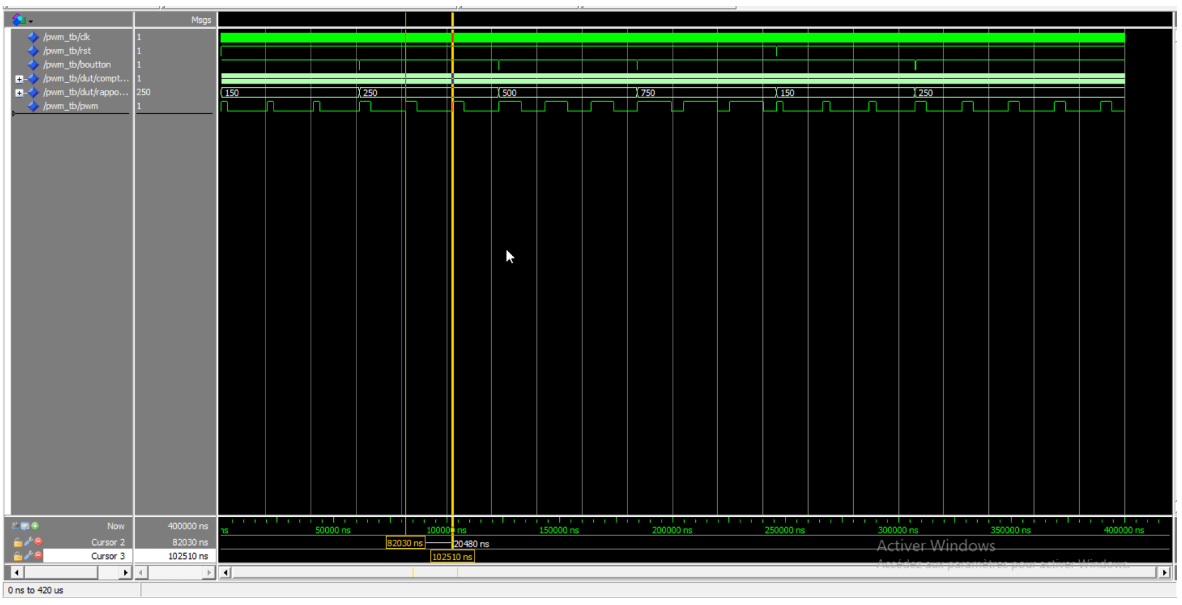


Figure: Simulation de fonction PWM

Sur la simulation, on constate que à chaque appuie sur Bouton, le rapport cyclique change. À l'aide des curseur, nous pouvons valider la fréquence de du signal. En effet, on observe un période de 20,48 us qui correspond à une fréquence de 48,82 kHz.

Après la simulation nous sommes passés sur le matériel.

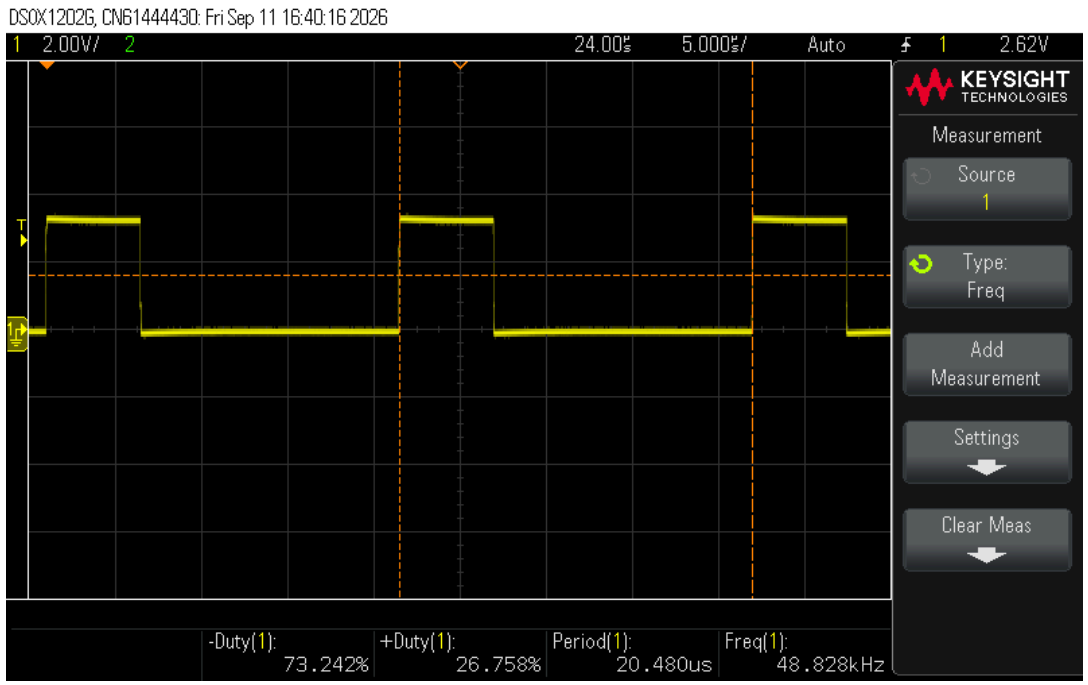


Figure 11 : visualisation du PWM à l'oscilloscope

L'observation du signal PWM à l'oscilloscope nous permet de confirmer l'étude faite précédemment. On constate que la fréquence est de 48,828 kHz.

F. Fonction supplémentaire

Cette partie, nous permet de développer les différentes fonctions supplémentaires qui était proposé au sein du TD mais qui n'était point obligatoire de réaliser. Nous avons pu nous entraîner via les fonctions décrites ci-dessous.

1. Décalage à droite

Ici, nous commençons par l'apprentissage des registres de décalage, nous commençons par le registre a décalage vers la droite.

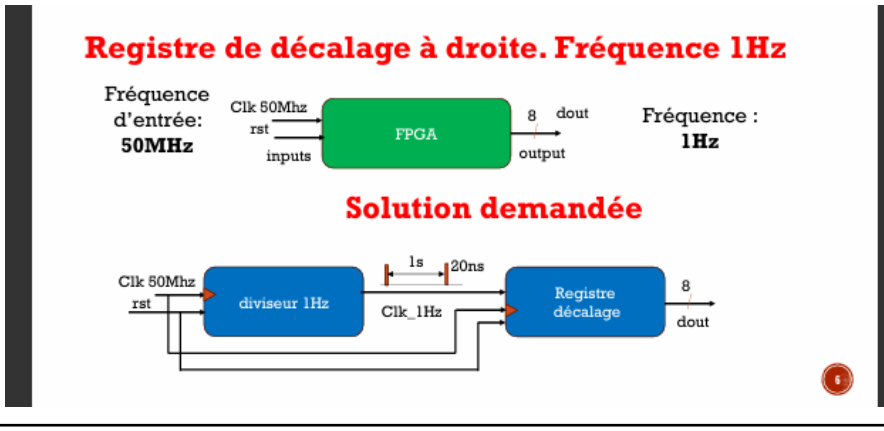


Figure 12 : Registre à décalage à droite

Pour réaliser le décalage à droite, on commence par initialiser une valeur binaire, par exemple « 0001 ». Ensuite, cette valeur est décalée vers la droite en effectuant une rotation des bits qui la composent. C'est-à-dire que pour « 0001 », lors d'un décalage à droite, on prend le bit de droite ('1') et on le replace devant les trois zéros, ce qui donne « 1000 », et ainsi de suite (« 0100 », « 0010 », « 0001 »).

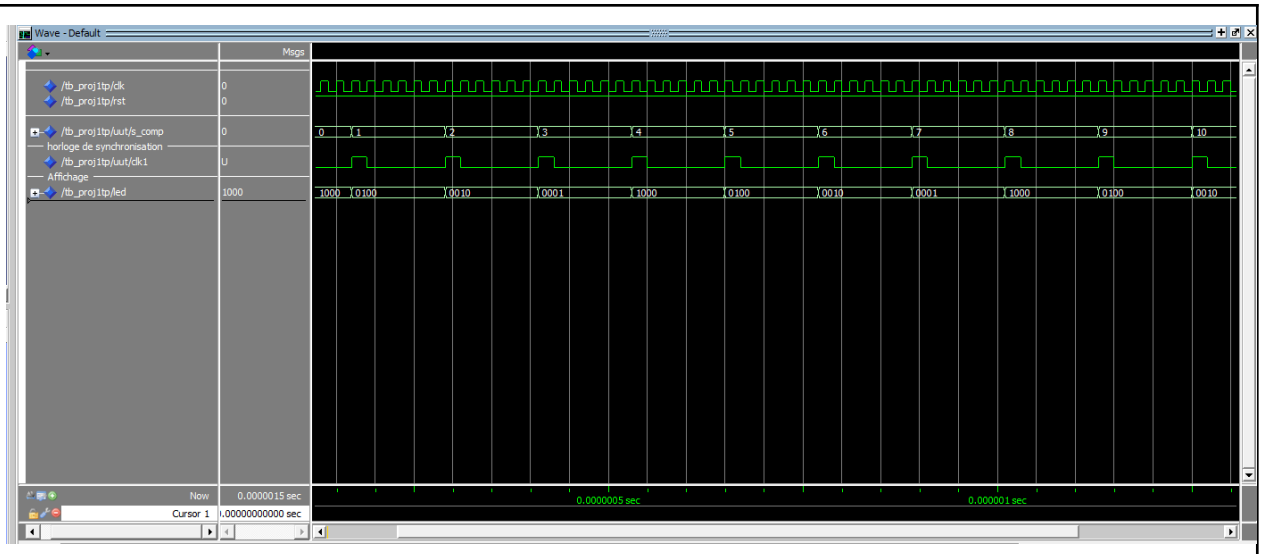


Figure 13 : Simulation

La simulation ci-dessus, nous montre bien le décalage expliqué juste avant. Comme demandé par la simulation, le décalage est rythmé sur la variable clk1 qui vient donner le top lors d'un de ses front montant.

2. Décalage contrôlé

Dans cette partie, nous allons faire le registre de décalage qui va être contrôlé par l'appuie d'un bouton.

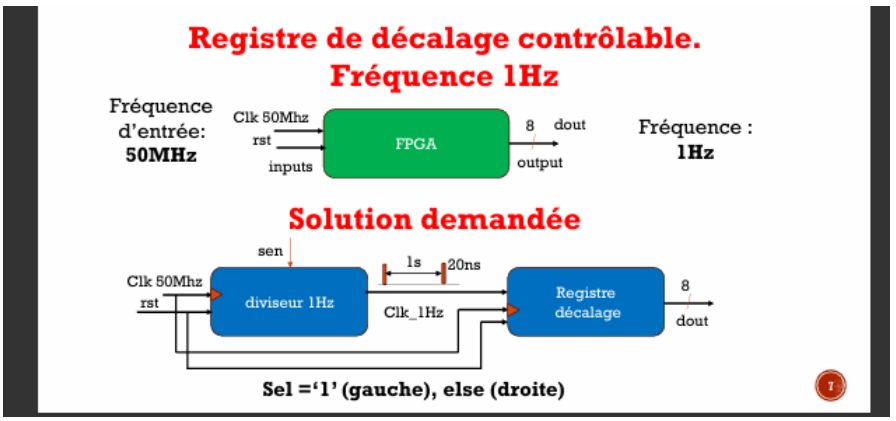


Figure 14 : Registre à décalage contrôlé

Afin de réaliser cette fonction, le système se comporte comme un registre de décalage vers la droite — de manière similaire à la description ci-dessus — tant que le bouton n'est pas pressé. Une fois le bouton actionné, la variable associée change d'état, ce qui enclenche le registre de décalage vers la gauche, selon le même principe mais dans la direction opposée. Par conséquent, tant que le bouton est maintenu, à chaque front montant de l'horloge `clk1`, le bit « 1 » est décalé vers la gauche, comme l'illustre la figure ci-dessous. Le sens du décalage est ainsi directement contrôlé par le biais de l'appui sur le bouton.

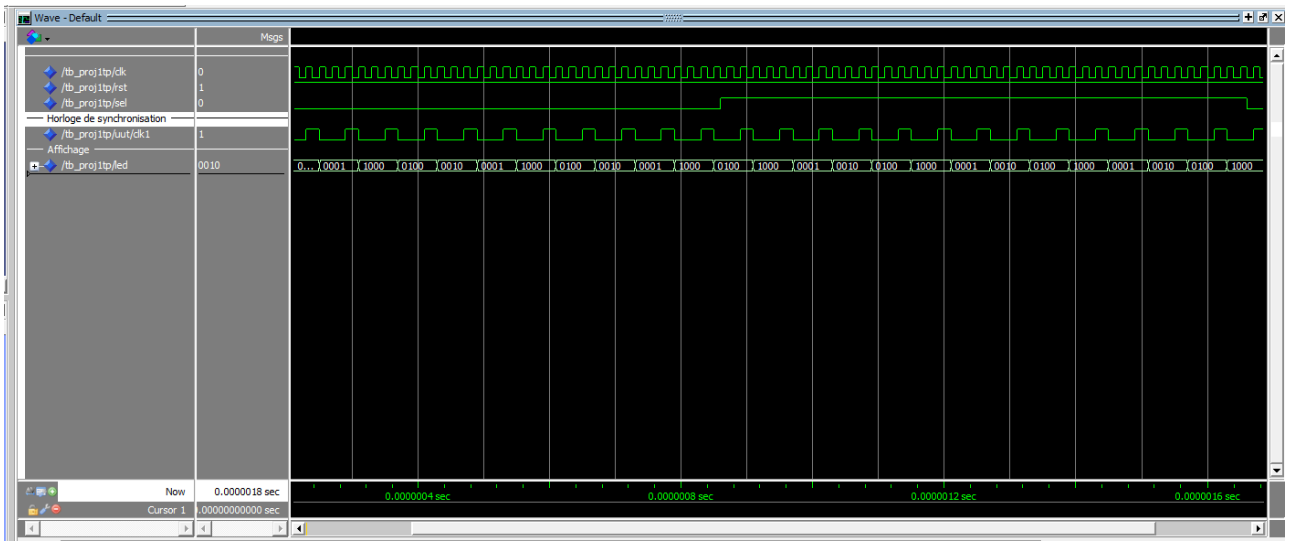


Figure 15 : Test bench

III. Fonction Simple (Cap)

Dans cette partie III, Nous devions réaliser une fonction simple afin de pouvoir diriger le bateau. Nous avions le choix entre deux fonctions, la première était l'anémomètre et la deuxième était le cap. Notre choix a été porté sur la fonction du cap. Le cap va nous permettre de gérer la direction du bateau via la direction de la boussole.

A. Étude fonctionnelle

L'entité **cap** mesure la durée du niveau haut du signal PWM délivré par la boussole électronique (in_pwm_compas) et la convertit en un cap exprimé en degrés, compris entre 0° et 359°. Le résultat est fourni sur 9 bits (data_compas) et accompagné d'un signal de validité (data_valid). Deux modes de fonctionnement sont proposés : un mode continu et un mode monocoup.

1. Interface

Entrées

Signal	Largeur	Description
clk_50M	1 bit	Horloge système 50 MHz
raz_n	1 bit	Remise à zéro, active à l'état bas

Signal	Largeur	Description
in_pwm_compas	1 bit	Signal PWM de la boussole, niveau haut de 1 ms à 36,9 ms
start_stop	1 bit	Commande de démarrage
continu	1 bit	Choix du mode : 1 = continu, 0 = monocoup

Sorties

Signal	Largeur	Description
data_compas	9 bits	Valeur du cap, de 0° à 359°
data_valid	1 bit	Indique que data_compas est valide
out_1s	1 bit	Sortie liée à la base de temps de 1 s

Le cap maximal (359) tient sur 9 bits, puisque $2^9 = 512$ valeurs sont codables.

2. Architecture

L'entité est composée de deux blocs, **Mode** et **Cap**, qui partagent l'horloge clk_50M et la remise à zéro raz_n.

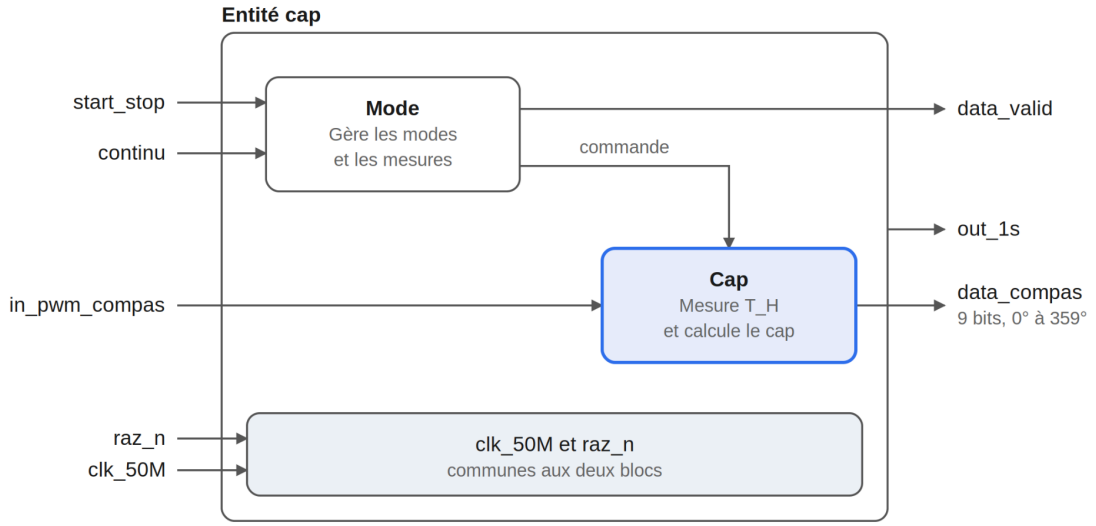


schéma bloc de l'entité cap · 2 blocs, 5 entrées, 3 sorties

Le bloc Mode reçoit start_stop et continu, commande le bloc Cap et fournit data_valid ; le bloc Cap reçoit in_pwm_compas et fournit data_compas.

Bloc	Entrées	Sorties	Rôle
Mode	start_stop, continu	commande vers Cap, data_valid	Gère les modes de fonctionnement et séquence les mesures

Bloc	Entrées	Sorties	Rôle
Cap	in_pwm_compas, commande du bloc Mode	data_compas	Mesure la largeur d'impulsion et calcule le cap

3. Caractéristiques du signal de la boussole

La boussole transmet le cap sous la forme de la largeur de l'impulsion haute T_H du signal in_pwm_compas :

- T_H = 1 ms correspond à un cap de 0° ;
- T_H = 36,9 ms correspond à un cap de 359°.

La relation entre T_H et le cap est linéaire : c'est la droite passant par les points (1 ms ; 0°) et (36,9 ms ; 359°).

4. Loi de conversion

La pente de la droite vaut :

$a = \frac{359}{36{,}1-1} = 10\ °/ms$

Le cap se déduit donc de la largeur d'impulsion par :

$data_compas = 10 \times (T_H - 1) = 10 \times T_H - 10$

avec T_H en ms et data_compas en degrés.

Vérification sur quelques points :

T_H (ms)	data_compas (°)
1	0
10	90
19	180
28	270
36,9	359

La résolution est de 1° pour 0,1 ms de variation de T_H.

5. Principe de mesure

La largeur T_H est mesurée par comptage :

1. Le front montant de in_pwm_compas déclenche le comptage.
2. Tant que in_pwm_compas reste à 1, un compteur (THB) est incrémenté à chaque période de la base de temps de mesure.
3. Le front descendant de in_pwm_compas arrête le comptage : T_H = THB × période de la base de temps.
4. Le cap est calculé avec la loi de conversion du paragraphe 5.

5. data_valid passe à 1 pour signaler que data_compas est disponible.

6. Modes de fonctionnement

- **Mode continu** (continu = 1) : les mesures sont enchaînées automatiquement(avec une fréquence 1Hz), et data_compas est mis à jour à chaque nouvelle mesure.
- **Mode monocoup** (continu = 0) : une seule mesure est effectuée, puis le système attend un front montant de start_stop pour en lancer une nouvelle.

La remise à zéro raz_n ramène le système à son état initial d'attente, avec data_valid = 0.

Après avoir analysé le cahier des charges et mener une étude fonctionnelle, nous avons établi la machine à état suivante pour modéliser le fonctionnement du cap et pouvoir l'implémenter facilement en VHDL.

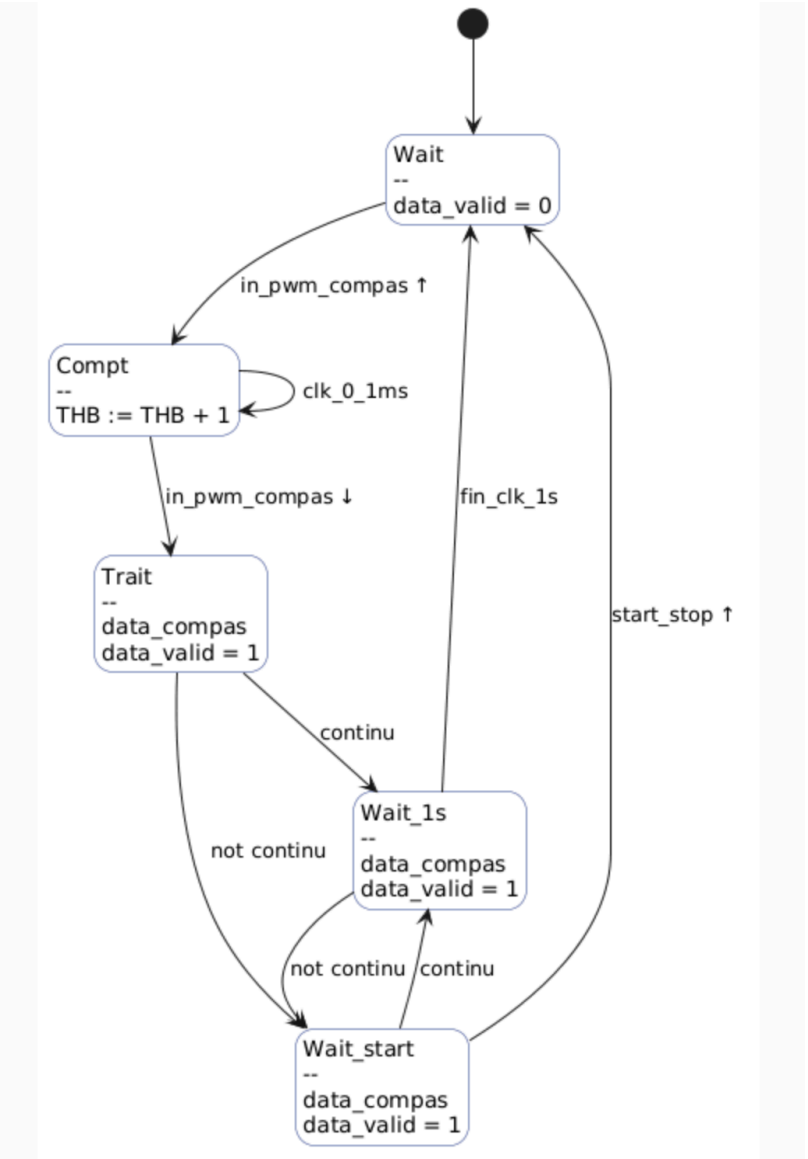


Figure 16 : Machine à état de la fonction Cap

B. Mise en œuvre en VHDL

Nous avons implémenté en VHDL. À l'aide de ModelSim, nous avons effectué des simulation pour valider le bon fonctionnement de la machine à état. Pour la simulation, nous avons fait un fichier testbench pour ne pas avoir à modifier manuellement les entrées. Ça se fera automatiquement selon le programme de test défini dans le testbench.

Le programme VHDL (cap.vhd) et son test bench associer sont sur le repo Git.

Les résultats de la simulation sont représentés ci-dessous. Les deux images représentent le résultat d'une seule simulation. Afin de pouvoir bien distinguer les différents modes, nous avons effectué en premier lieu une analyse pour continu=0 (monocoup) puis pour le mode continu.

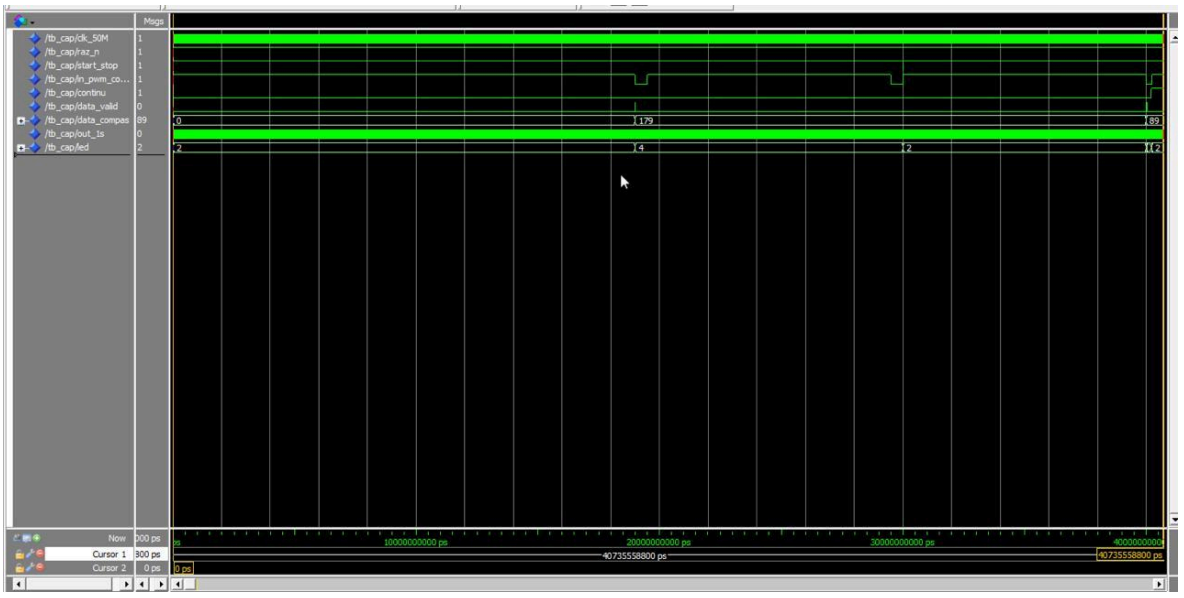


Figure : Mode monocoup (continu =0)

Sur cette figure nous pouvons analyser le comportement en mode monocoup.
On constate bien que ce mode est actif quand continu=0. Dans ce cas, l'acquisition de mesure de l'angle (data_compas) est déclenchée lorsque le signal start_stop passe à "0"(état actif). Les résultats de la simulation confirment ce fonctionnement. En effet, sur la simulation, on observe qu'il y a eu deux appuis sur le bouton start_stop et deux acquisition de mesure alors qu'il y a trois PWM qui ont été générées. Ce qui justifie qu'un PWM n'a pas été pris en compte car start_stop n'est pas actif. C'est le deuxième PWM sur la figure ci-dessus.

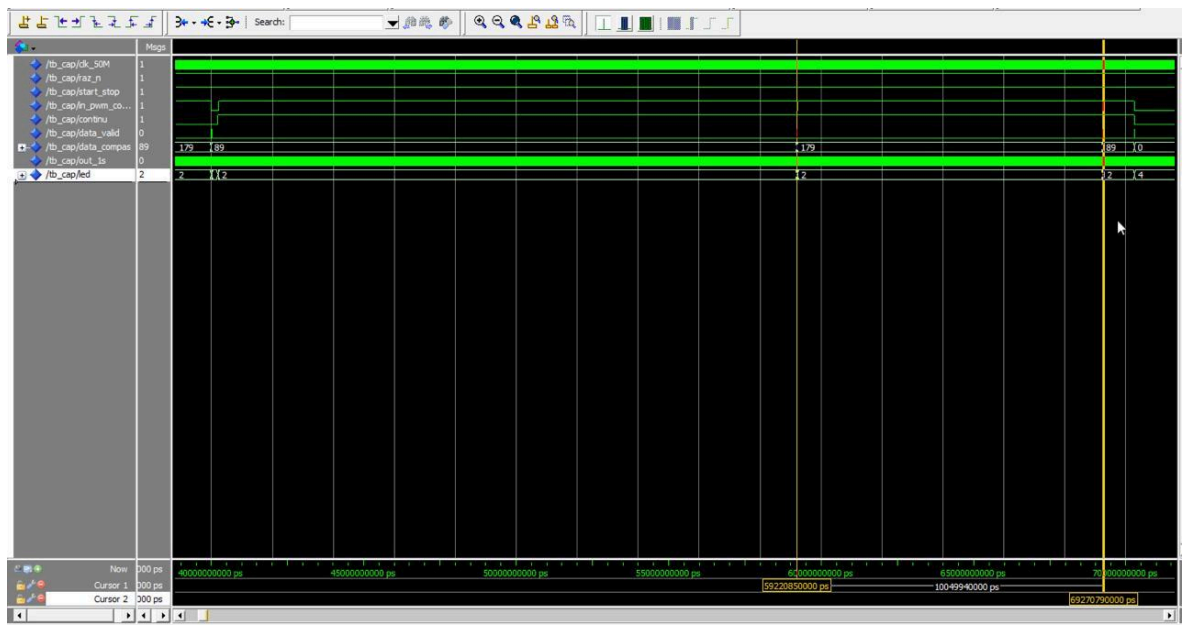


Figure: Mode continu (continu = 1)

En mode continu, l'acquisition de mesure de l'angle s'effectue de façon périodique avec un période de 1 seconde. Ce mode est actif lorsque "continu=1". La figure ci-dessus illustre le fonctionnement de ce mode. On constate que l'acquisition s'effectue sans que le bouton "start_stop" ne soit appui. Pour mettre en évidence la période d'acquisition, nous avons utilisé des curseurs pour déterminer l'intervalle entre deux acquisitions. Comme l'indique la figure, nous avons 1 seconde entre les deux curseurs qui ont été positionnés sur deux mesures successives.

IV. Fonction complexe

Le choix n'a pas encore été fourni.

V. Conclusion

VI. Annexe